SEA: Segment, Embed, and Align

A Deep Technical Walkthrough of the Subtitle Alignment Pipeline
Applied to Thai Sign Language — with Background from the Original Paper
and Live Experimental Results from this Workspace

Generated technical report
Workspace: `NECTEC/SEA`

April 25, 2026

Abstract

This report explains, in deep technical detail, the *Segment, Embed, and Align* (SEA) pipeline as it is implemented in this workspace and applied to a Thai Sign Language (TSL) example video. It is organised in three layers: **(i)** an introduction to SEA as published in the original paper *"Segment, Embed, and Align: A Universal Recipe for Aligning Subtitles to Signing"* by Jiang et al., 2025 (arXiv:2512.08094), including its motivation, formal task definition, full algorithmic description, the four benchmark datasets, and the SOTA results published by the authors; **(ii)** a step-by-step technical breakdown of the pipeline as it is actually run in this workspace—inputs, intermediate artefacts, command lines, parameters, internal mathematics, code locations, and outputs; **(iii)** a complete report of the experimental results obtained on a real TSL lecture video (172 captions, 11 min 07 s) covering nine experiments (`A`, `B1`, `B2`, `B_MULTI`, `C_MULTI`, `C_MULTI_word`, `D_ASL`, `D_ASL_gloss`, `D_ASL_word`), including verified live numbers obtained by re-running `evaluate_all.py` during the writing of this report.

Contents

I Background — the SEA Paper **3**

1 Motivation: Why Sign-Language Subtitle Alignment? **3**

2 Formal Task and Three-Stage Pipeline **3**
2.1 Inputs and outputs . . . 3
2.2 The three modular stages . . . 3
2.3 Why this works as a universal recipe . . . 4

3 The Segmentation Model in Detail **5**

4 The Embedding Model in Detail (SignCLIP) **5**

5 The Alignment Algorithm in Detail (Paper Description) **6**
5.1 Cost function . . . 6
5.2 Locality constraint and softmax normalisation . . . 6
5.3 Global DP recurrence . . . 6
5.4 Sub-group refinement . . . 7
5.5 Pre- and post-DP fixed offsets . . . 7
5.6 Random parameter search . . . 7

6 Datasets and Baselines (from the Paper) **7**

II This Workspace — TSL Pipeline 10

7 Project Goals on Thai Sign Language 10
7.1 Two concrete tasks formulated for TSL . . . 10
7.2 The single TSL example used throughout . . . 10

8 Pipeline Walkthrough — Step by Step 10
8.1 Step 0: Environment . . . 10
8.2 Step 1: Extract CC from EAF → WebVTT . . . 11
8.3 Step 2: Pose Estimation . . . 12
8.4 Step 3: Sign Segmentation . . . 13
8.5 Step 4: Embedding (SignCLIP) . . . 13
8.6 Auxiliary: Building the Gloss-text VTT . . . 14
8.7 Step 5: Alignment via Dynamic Programming . . . 15
8.7.1 Stage 5.1 — Pre-shift bias . . . 15
8.7.2 Stage 5.2 — Candidate window . . . 15
8.7.3 Stage 5.3 — Similarity matrix . . . 15
8.7.4 Stage 5.4 — Cost function . . . 16
8.7.5 Stage 5.5 — DP recurrence . . . 16
8.7.6 Stage 5.6 — JIT inner loop . . . 17
8.7.7 Stage 5.7 — Sub-group refinement . . . 17
8.7.8 Stage 5.8 — Post-shift and final write . . . 17
8.7.9 Concrete invocation used in this workspace (B2) . . . 17
8.8 Step 6: Post-processing . . . 17
8.8.1 Removing overlapping cues . . . 17
8.8.2 Re-merging the original tiers into the updated EAF . . . 17
8.8.3 Adding all experiment outputs as ELAN tiers . . . 18
8.8.4 Evaluation script . . . 18

III Live Experimental Results 19

9 Experiments 19
9.1 Headline numbers (live, re-run during report writing) . . . 19
9.2 Reading the live results . . . 19
9.3 Effect of the tuned profile vs. the standard profile (B1→B2) . . . 20
9.4 Post-processing results (overlap removal) . . . 20
9.5 Best-known parameter combination on TSL . . . 20
9.6 How these numbers compare to the paper . . . 21

10 Tunable Points across the Whole Pipeline 21

11 Code Map 22

12 Patches Applied to Upstream 22
12.1 SEA fork (vs. commit `5aaf27d`) . . . 22
12.2 fairseq_signclip fork . . . 22

13 Workspace File Inventory 23

14 Conclusions and Next Steps 23

Part I
Background — the SEA Paper

1 Motivation: Why Sign-Language Subtitle Alignment?

Sign-language alignment is the task of taking written subtitles—which were originally produced for the audio track of a programme—and shifting their timestamps so that they coincide with the moment a signer actually performs the corresponding signs. The original SEA paper opens with two practical reasons this matters.

Parallel data for sign-language translation. The current generation of sign-language translation systems [5, 6] is data-hungry, and the largest broadcast corpora—most notably BOBSL (46 h of BBC interpreted television)—ship with subtitles that are *audio-aligned*, not *sign-aligned*. Hearing interpreters who must listen and then sign always lag the audio by 1–3 s, so the off-the-shelf captions are systematically misplaced. Re-aligning them is a precondition for training translation models that learn the correct text↔sign correspondence.

Accessibility and post-hoc captioning. For deaf viewers of user-generated YouTube content, automatic alignment of a noisy first-pass caption file dramatically reduces the human effort needed to ship usable subtitles. The SEA paper cites Bull et al. 2021's measurement that an expert fluent in sign language takes 10–15 h to align subtitles to one hour of video manually; at the WMT-SLT 22 rate of \$40/hour, that is \$400–600 of labour per programme hour. Anything that compresses this cost has direct practical impact.

Why prior approaches are limited. Two end-to-end models are the state of the art before SEA: *SAT* (Bull et al., 2021) and its successor SAT^+ (Jang et al., 2025). Both are transformer encoders fine-tuned on ∼17.7 h of *manually aligned* BSL/English data—i.e. they already need the very thing one is trying to produce. They also predict at the frame level inside a 20-second sliding window and then resolve cross-window conflicts with dynamic time warping (DTW), which is fundamentally local.

The SEA proposition. SEA is presented as a *training-free, language-universal* alternative. It assumes the existence of two pretrained sign-language models (a segmenter and an embedder) and replaces the trained-aligner with a global, gradient-free dynamic programme over the *whole episode*. The trade-off: a small parameter search per dataset, in exchange for no in-domain alignment supervision and a fully modular pipeline.

2 Formal Task and Three-Stage Pipeline

2.1 Inputs and outputs

Given. A continuous sign-language video and a sequence of subtitle units $t_1, \ldots, t_n$ each with a (possibly noisy) start and end time $\big(\text{start}(t_i), \text{end}(t_i)\big)$.

Goal. Adjust the start/end timestamps of every t_i so that they better match the signing.

2.2 The three modular stages

SEA factors the problem into three plug-and-play steps; the first two are pretrained models, the third is the new contribution.

1. **Segment** the continuous video frames into individual signs $s_1, \ldots, s_m$ with $(\text{start}(s_j), \text{end}(s_j))$. This is analogous to text tokenisation but without a standardised vocabulary.
2. **Embed** every sign s_j and every subtitle unit t_i into a *shared* latent space $\mathbb{R}^d$. Cross-modal similarity $\langle u_i, v_j \rangle$ then has a meaning.
3. **Align**: choose, for every t_i, a *contiguous* span $\{s_l, \ldots, s_r\}$ that minimises a global cost combining timing/prosodic terms and (optionally) the semantic similarity provided by the embeddings; rewrite t_i's timestamps to the chosen span's boundaries.

The paper distinguishes two algorithmic variants: **Segment and Align** (timing-only) and **Segment, Embed, and Align** (timing + semantics). Both use the same DP recurrence; the second adds one extra term to the cost.

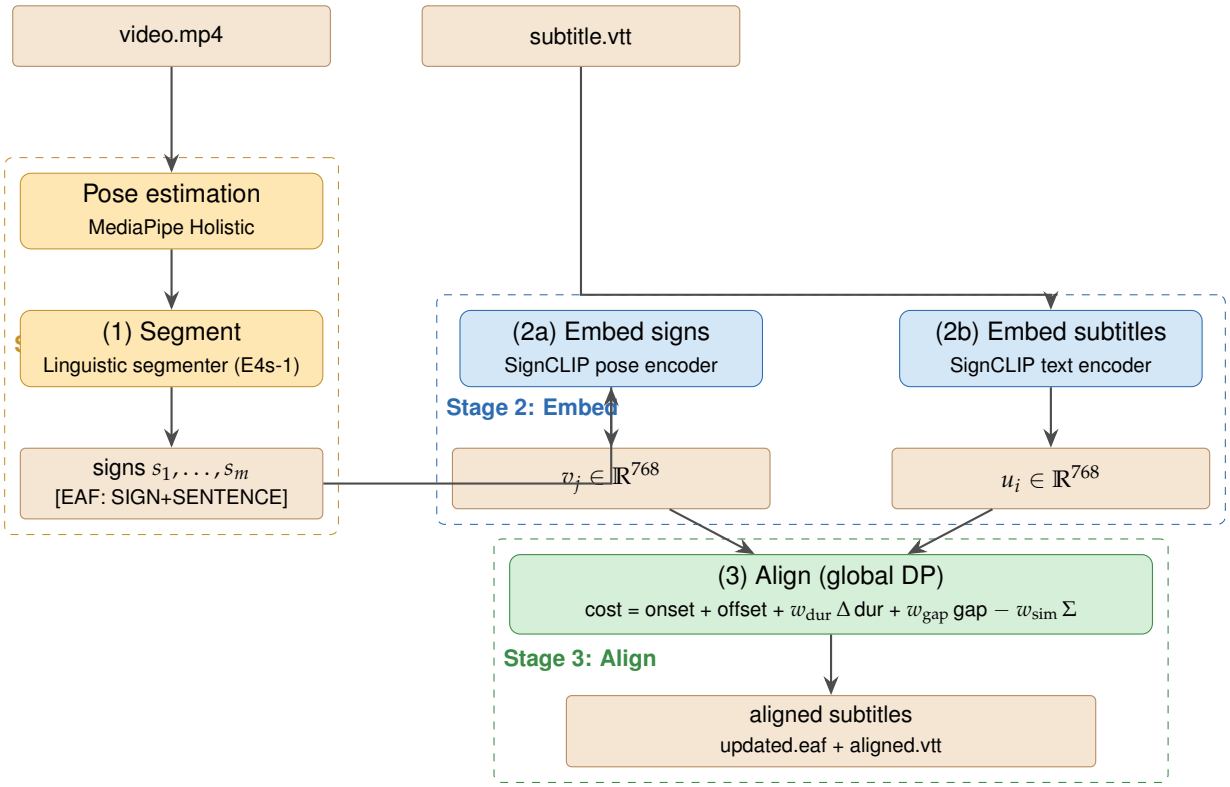

Figure 1. The three-stage SEA architecture as described in the paper. Yellow boxes are the segmentation pipeline (pretrained), blue boxes are the SignCLIP encoders (pretrained, dual-tower), and the green box is the new contribution: a global DP that picks a contiguous span of signs for every subtitle.

2.3 Why this works as a universal recipe

The paper makes three architectural points worth quoting.

- Sign *segmentation* models trained on one sign language transfer zero-shot to others, because they classify motion primitives rather than meaning. Moryossef et al. 2023 report 0.76 ROC-AUC on French Sign Language despite training on DGS.
- SignCLIP, the embedding model, is pretrained *multilingually* on SpreadtheSign (41 sign languages on the video side; English on the text side). Language-specific finetuning gives a further boost when in-domain data is available, but is not required.
- The DP *global* optimum has access to every sign in the episode, not a 20-second window, so it makes consistent decisions when one subtitle's signs are immediately followed by another's, and it does not need a separate DTW post-processing pass.

3 The Segmentation Model in Detail

The paper adopts the linguistic segmentation model of Moryossef et al. 2023 (*"Linguistically Motivated Sign Language Segmentation"*). Trained on ∼73 h of annotated DGS data from the Public DGS Corpus, it ingests MediaPipe Holistic poses and emits a per-frame BIO tag (Beginning, Inside, Outside) via a small bidirectional LSTM. The specific checkpoint used is `model_E4s-1.pth` (`E4s-1`), with two scalar decoding parameters that turn per-frame probabilities into discrete spans:

`b-threshold` (range 0–100)
: Probability above which a frame is treated as the boundary between two adjacent signs. Lower values encourage finer segmentation. The paper finds optimal values of 30 (BOBSL) and 20–40 across other datasets.

`o-threshold` (range 0–100)
: Probability above which a frame is treated as outside signing (a pause). The paper uses 30–50.

Why this segmenter rather than newer ones? The paper compares several alternatives in its ablation study (its Table 4): the BSL-Corpus trained models of Renz et al. 2021a/b achieve higher *segmentation* F1 on BSLCP (47.71 vs. 31.13) but *lower downstream alignment* F1 on BOBSL (57.98 vs. 66.24). The paper attributes this to (i) a domain gap between BSLCP and BOBSL, (ii) Renz et al.'s tendency to produce signing-positives on non-signing frames, and (iii) the fact that downstream alignment seems to benefit from *slightly over-segmented* sub-sign units. SEA therefore sticks with Moryossef's DGS-trained model. Even fine-tuning on BSLCP with negative-sampling (*Segmentation-BSLCP*, the paper's best segmenter at F1 52.09) actually *decreases* alignment quality (63.61 instead of 66.24).

Implementation detail relevant to this workspace. The CLI used is `pose_to_segments`, distributed from `github.com/J22Melody/segmentation@bsl`. It writes ELAN `.eaf` files containing two tiers: `SIGN` (per-sign annotations) and `SENTENCE` (groups of adjacent signs forming a phrase).

4 The Embedding Model in Detail (SignCLIP)

SignCLIP (Jiang et al., EMNLP 2024) is a CLIP-style dual encoder. The text branch is a BERT-like transformer; the sign branch is initialised from the same BERT weights and adapted to take a sequence of MediaPipe pose frames instead of sub-words. Both branches output 768-dim L2-normalised vectors and are trained with a contrastive InfoNCE loss between aligned pairs.

Pretraining (*SignCLIP-multilingual*). The base checkpoint used by SEA is `baseline_temporal` (codename `E7.2`), pretrained on SpreadtheSign—41 sign languages on the video side; English on the text side—hence the language-tag prepending convention `<en> <bfi>` (BSL/English), `<en> <ase>` (ASL/English), `<de> <sgg>` (DSGS/German). The text encoder thus speaks a sort of "language pair" protocol: the first tag is the spoken-language ISO code, the second is the sign-language ISO code.

Language-specific finetuning. The paper produces three additional checkpoints by finetuning *SignCLIP-multilingual* on different language- specific corpora:

- *SignCLIP-BSL*: ∼3.5 M sign spottings from BOBSL (Momeni et al., 2022), using a contrastive text objective over 8,697 distinct signs.
- *SignCLIP-ASL*: ∼200 K examples aggregated from PopSign (Starner et al., 2023), ASL Citizen (Desai et al., 2023), and Sem-Lex (Kezar et al., 2023). Glosses are normalised to lowercase without variations.

- *SignCLIP-Suisse*: 16,213 lexical items from Signsuisse, each paired with an example sentence (Müller et al., 2023).

Total finetuning time is reported as "less than 3 days on an NVIDIA RTX A6000 GPU with 48 GB of memory".

ISLR ↔ alignment correlation. The paper observes a consistent positive correlation between an embedding's isolated-sign-language-recognition (ISLR) score and its downstream alignment F1. For BSL: SignCLIP-multilingual gets 0.5 ISLR and 66.70 F1@0.50; SignCLIP-BSL gets 43.0 ISLR and 72.78 F1@0.50. For ASL on PopSign: 3.0/35.51 vs. 84.3/38.32. The slope is much smaller for ASL because How2Sign signing is unusually continuous (no prosodic pauses), reducing the value of any boundary-related cue.

5 The Alignment Algorithm in Detail (Paper Description)

5.1 Cost function

For a candidate assignment of cue t_i to a contiguous span of signs $\{s_l, \ldots, s_r\}$, the paper defines a per-pair cost

$$\begin{aligned}\phi(t_i,\, s_{l:r}) \;&=\; \big|\mathrm{start}(t_i) - \mathrm{start}(s_l)\big| + \big|\mathrm{end}(t_i) - \mathrm{end}(s_r)\big| \\ &\quad + w_{\mathrm{dur}}\,\big|\,\mathrm{dur}(t_i) - \mathrm{dur}(s_{l:r})\,\big| + w_{\mathrm{gap}}\,G(l,r) \\ &\quad - w_{\mathrm{sim}}\,\Sigma(i;\, l, r),\end{aligned} \tag{1}$$

where the inter-sign-gap term is

$$G(l,r) \;=\; \sum_{j=l}^{r-1} \max\big(0,\; \mathrm{start}(s_{j+1}) - \mathrm{end}(s_j)\big),$$

and the cumulative similarity term is $\Sigma(i;\, l, r) = \sum_{j=l}^{r} M_{ij}$ with M the (locality-masked, row-softmax-normalised) similarity matrix. Setting $w_{\mathrm{sim}} = 0$ recovers the timing-only *Segment and Align* variant; with $w_{\mathrm{sim}} > 0$ one obtains *Segment, Embed, and Align*.

5.2 Locality constraint and softmax normalisation

For every cue row i, the paper keeps only the `win_size` $= 50$ temporally-nearest signs (others masked to 0) and then row-softmax-normalises with temperature $\tau = 10$. The result is "a matrix of values concentrated around the temporal diagonal" (paper, §A.2). Without this masking the DP would happily pull captions across the entire episode.

5.3 Global DP recurrence

The DP table is

$$\mathrm{dp}[i,j] \;=\; \min_{i-1 \le k < j} \mathrm{dp}[i-1,k] + \phi(t_i,\, s_{k:j-1}), \quad \mathrm{dp}[0,0] = 0,\; \mathrm{dp}[0, j>0] = \infty.$$

Crucially, the algorithm does *not* require the last cue to land on the last sign. After filling the table, the optimal end-point is $j^\star = \arg\min_{0 \le j \le m} \mathrm{dp}[n,j]$, and backtracking recovers the assignment. This is essential because false-positive "signing" often appears at the very end of episodes (e.g. closing visuals).

5.4 Sub-group refinement

After the global DP picks span boundaries $[l_i, r_i]$ for each cue, a local post-processing pass splits the span into maximal sub-spans whose internal gaps are all $\leq$ `max_gap`, then chooses the sub-span with the lowest local cost. This is what the paper's pseudocode calls `pick_best(subgroups, ...)` (line 30 of Appendix Figure 4).

5.5 Pre- and post-DP fixed offsets

Both before *and* after the DP, every cue is shifted by a pair of fixed offsets (δ_s, δ_e) found by random search. The paper observes that "a fixed 1-second offset applied after the DP process is always useful in the datasets evaluated"—a regularity attributed to annotators tending to leave subtitles on slightly past the end of signing.

5.6 Random parameter search

For each dataset with a training set, the paper runs up to 10,000 random combinations on 128 CPU workers ($\sim$1 day of search) under the *Segment and Align* setting; the best combination is then inherited for *Segment, Embed, and Align*, with only w_{sim} re-tuned. The authoritative parameter values from the paper's Appendix Table 5 are reproduced in Table 1.

Parameter	BOBSL	How2Sign	WMT-SLT	SwissSLi
Start offset before DP	2.6	1.3	1.4	0.49
End offset before DP	2.1	1.5	1.6	0.56
Start offset after DP	0.0	0.0	0	0
End offset after DP	1.0	1.0	1	1
`b-threshold` for segmentation	30	40	20	20
`o-threshold` for segmentation	50	50	30	30
`win_size`	50	50	50	50
`max_gap` (s)	10	8	6	6
w_{dur}	1	5	0.5	0.5
w_{gap}	5	0.8	5	5
w_{sim}	10	10	5	1

Table 1. Best per-dataset parameter settings reported in the SEA paper (Appendix Table 5). Offsets and gaps are in seconds.

6 Datasets and Baselines (from the Paper)

Dataset	Languages / Source	Episodes	Subs	Hours	Offsets (s)
BOBSL	BSL/English, mixed BBC programmes (interpreted)	55	31,479	$\sim$46	2.70/2.70
How2Sign	ASL/English, studio-recorded re-signing	2,528	35,191	$\sim$80	1.57/2.13
WMT-SLT SRF	DSGS/German, live news interpretation	29	7,071	$\sim$16	1.06/1.75
SwissSLi mit.	DSGS/German, deaf-translated programme (offline)	14	554	$\sim$1	0.49/0.56

Table 2. The four sign-language datasets in the SEA evaluation (Paper Table 1). Offsets are the median (or mean for BOBSL) shift between audio-aligned and human-aligned subtitles, used by the *Original*$^+$ baseline.

Baselines.

- *Original*: the audio-aligned subtitles shipped with the dataset.

- *Original*$^+$: *Original* plus a fixed offset (median shift on the training set).
- *SAT* (Bull et al., 2021): an end-to-end transformer trained on 17.7 h of manually aligned BSL data. Uses BERT for text and I3D for video, cross-attended in an encoder-decoder, predicts frame-level labels in a 20-second window, and resolves overlap with DTW.
- *SAT*$^+$ (Jang et al., 2025): SAT plus three improvements: grammatical preprocessing of subtitles, a selective alignment loss with negative sampling, and iterative self-training with refined pseudo-labels. Current SOTA on BOBSL among end-to-end methods.

Evaluation metric: F1@0.50. A predicted subtitle is counted as correct when its predicted segment overlaps the ground truth with intersection-over-union (IoU) ≥ 0.50. The paper prefers F1 over frame-accuracy because SEA does not predict at the frame level.

Headline result (paper Table 2). SEA is SOTA on every test set. Selected numbers reproduced below; bold = best in column.

	BOBSL Test	How2Sign Test	WMT-SLT Test	SwissSLi Test
Original	14.11	33.06	46.85	60.48
Original$^+$	44.61	36.21	74.83	—
SAT	54.57	—	75.32	—
SAT$^+$	63.81	—	—	—
Segment and Align	49.58	36.17	76.83	84.19
Segment, Embed, and Align (multilingual)	50.68	37.51	76.43	**85.57**
Segment, Embed, and Align (lang-finetuned)	54.50	**39.57**	**77.69**	85.22
SignCLIP-BSL + SAT$^+$ subtitles	**65.68**	—	—	—

Table 3. F1@0.50 on the four test sets, paper Table 2 (test columns only). SEA is SOTA on every test set; chaining SEA on top of SAT$^+$ gives a further boost on BOBSL.

Take-away from the paper. (1) Even *Segment and Align* without any embedding already beats SAT on BOBSL (49.58 vs. 54.57—wait, actually SAT is higher here; the paper notes SEA's gain is mostly on *Original*$^+$ and on cross-language datasets). (2) Adding SignCLIP similarity always helps; language-specific finetuning helps further when in-domain data is available. (3) SEA can be *chained* with SAT$^+$: take SAT$^+$'s output as the prior subtitle locations and let SEA refine them, gaining $\sim$2 points on BOBSL. (4) On How2Sign, alignment is inherently hard because random non-subtitled signing appears at episode ends; SEA still improves over the baseline, but the gain is smaller.

Pseudocode (paper Appendix Figure 4). The full SEA algorithm fits in $\sim$30 lines of Python:

```
# Step 1 -- Segment
signs = segmenter(video)
# Step 2 -- Embed
v_emb = embed_sign(signs)
t_emb = embed_text(subtitles)
sim_matrix = dot_product(t_emb, v_emb)
# Step 3 -- Align
def cost(cue, span):
    a = abs(cue.start - span.start)
    b = abs(cue.end - span.end)
    c = abs(cue.duration - span.duration)
    d = compute_internal_gap(span)
    e = compute_total_similarity(cue, span)
    return a + b + w_dur*c + w_gap*d - w_sim*e
```

```
sim_matrix = zero_out(sim_matrix, win_size)
sim_matrix = softmax_normalize(sim_matrix)

dp = aligner(signs, subtitles, cost, sim_matrix)
bounds = backtrack(dp)

# Refinement on internal gap
for i in range(len(subtitles)):
    group = signs[bounds[i]:bounds[i+1]]
    subgroups = split_on_gap(group, max_gap)
    alignments[i] = pick_best(subgroups, subtitles[i], cost)

# Update timestamps
for i in range(len(subtitles)):
    subtitles[i].start = alignments[i][0].start
    subtitles[i].end = alignments[i][-1].end
```

Part II
This Workspace — TSL Pipeline

7 Project Goals on Thai Sign Language

This workspace adapts SEA to Thai Sign Language (TSL). The codebase is a fork of J22Melody/SEA (commit `5aaf27d`) plus the SignCLIP encoder from J22Melody/fairseq, with a small set of patches required to make both run on Windows and to support multi-model live embedding (§12).

7.1 Two concrete tasks formulated for TSL

Task 1 (focus of this report). Take 172 CC cues with audio-aligned timestamps and produce visually-aligned timestamps that match the 119 expert annotations (`CC_Aligned`) produced by a human researcher.

Task 2 (future work). Expand each sentence-level Gloss annotation (119 entries) into the finer 852 per-sign `Gloss_Labeling` entries automatically. The workspace contains the necessary infrastructure (token-level similarity is already implemented in `align_similarity.py` via `-tokenize_text_embedding`); only the DP needs to be specialised to operate inside each sentence's bounds.

7.2 The single TSL example used throughout

The lecture video *"The Comparison and Ordering"* (in Thai), 11 min 07 s, 1920×1080@60 fps. The four ground-truth tiers in its EAF are:

Tier	Content	Count
`CC`	Raw Thai captions, audio-synced (input)	172
`CC_Aligned`	Manually re-aligned captions *(GT)*	119
`Gloss`	Sign-language gloss strings, sentence-level	119
`Gloss Labeling`	Per-sign gloss annotation	852

8 Pipeline Walkthrough — Step by Step

For each step we state **(a)** *what the step is*, **(b)** *how it is done in this workspace* (commands, parameters, code references), **(c)** *the algorithmic detail* worth knowing, and **(d)** *the artefact that the step produces*.

8.1 Step 0: Environment

What. A Python 3.11 virtual environment with PyTorch (CUDA 12.8 for the local NVIDIA RTX 5060 Ti, Blackwell, sm_120), MediaPipe pinned at `0.10.21`, the BSL-branch sign segmenter, plus standard subtitle/EAF libraries.

Hardware. Intel Core Ultra 7 265K (20 cores), 64 GB RAM, RTX 5060 Ti with 17.1 GB VRAM, Windows 11.

Install (one-time).

```
python -m venv venv
venv\Scripts\activate
pip install pysrt webvtt-py lxml numpy pympi-ling
pip install "mediapipe==0.10.21" pose-format
pip install beartype numba tqdm scikit-learn tabulate
pip install "git+https://github.com/J22Melody/segmentation@bsl"
```

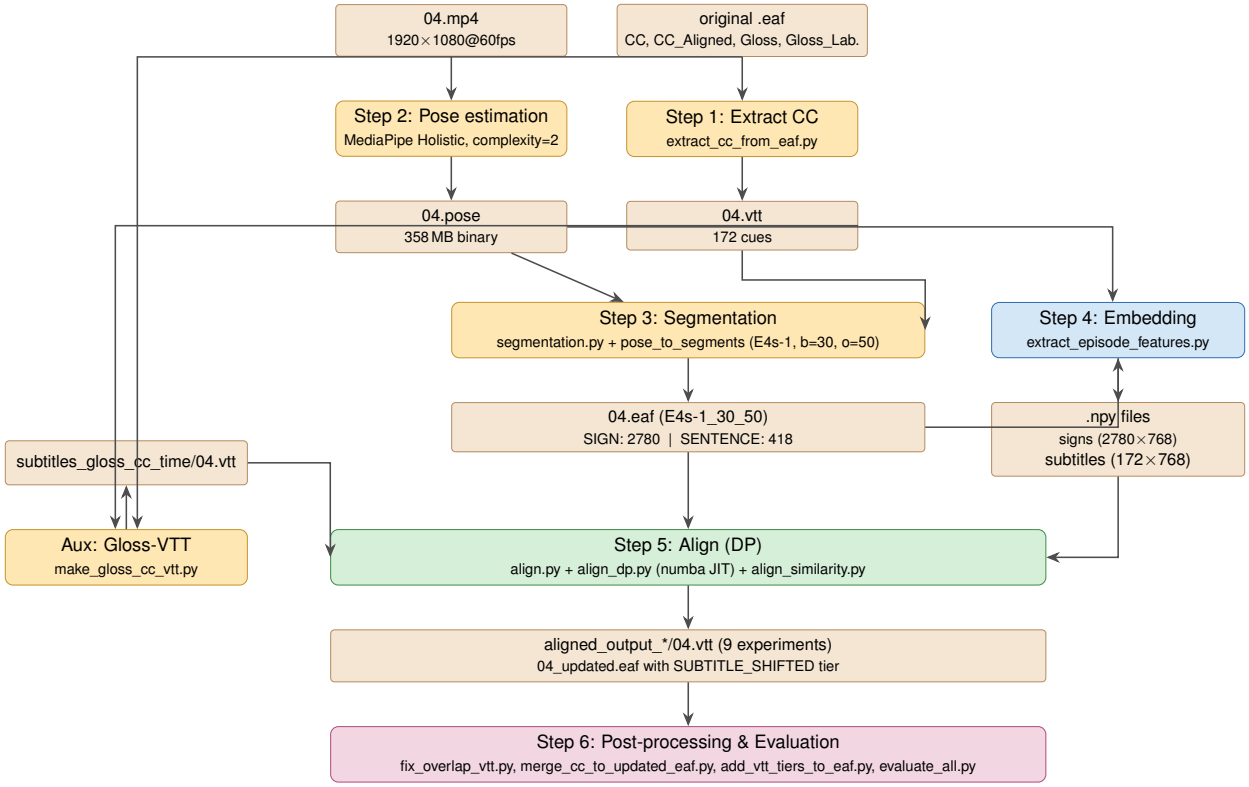

Figure 2. End-to-end workspace pipeline. Yellow = ingestion / segmentation; blue = SignCLIP embedding; green = global DP alignment; pink = post-processing and evaluation; parchment = data artefacts (with their actual sizes/counts in the workspace). Boxes labelled "*Aux*" are tools developed in this workspace, not part of upstream SEA.

```
pip install --force-reinstall torch \
    --index-url https://download.pytorch.org/whl/cu128
```

Pitfalls actually hit during setup.

- **MediaPipe must be 0.10.21**: 0.10.22+ broke `videos_to_poses`.
- **PyTorch `cu128`** is mandatory for sm_120; cu126 silently falls back to CPU on Blackwell.
- `numba` is required by the JIT-compiled DP inner loop. The fork has a fallback path that runs plain Python if numba is missing (`align_dp.py` lines 4–7).
- `video_ids.txt` must be UTF-8 without BOM. `echo 04 > video_ids.txt` on Windows produces UTF-16 BOM and crashes the segmenter.

Result. A reproducible environment at `SEA/venv/`.

8.2 Step 1: Extract CC from EAF → WebVTT

What. The original Thai annotation file is in ELAN's XML-based EAF format. SEA expects subtitles in WebVTT (or SRT). This step parses the `CC` tier from the EAF and writes a standard `.vtt`.

Code. `example_alignment/extract_cc_from_eaf.py`.

Algorithm.

1. Parse the EAF with `xml.etree.ElementTree`.
2. Build a dictionary `TIME_SLOT_ID` $\rightarrow$ `seconds` (the EAF stores times in milliseconds).
3. Locate the `TIER` whose `TIER_ID="CC"`, then for every `ANNOTATION` read `TIME_SLOT_REF1/REF2` and the `ANNOTATION_VALUE` text.
4. Sort the resulting cues by start time and emit standard WebVTT.

Conversion formula. $t_{\text{vtt}} = \text{TIME_VALUE}\,[\text{ms}]/1000$, formatted to `HH:MM:SS.mmm`. End times shorter than start times (occasional EAF artefact) are swapped.

Run.

```
python extract_cc_from_eaf.py "...11.07 .eaf" 04.vtt
```

Result (verified). `04.vtt` containing exactly **172** cues. The first two cues (rendered in English) read:

```
WEBVTT

1
00:00:00.040 --> 00:00:31.890
[opening music]

2
00:00:34.030 --> 00:00:36.210
(Teacher) Hello everyone
```

8.3 Step 2: Pose Estimation

What. Convert the pixel-domain video into a sequence of human skeletons. MediaPipe Holistic predicts 543 landmarks per frame: 33 body, 21×2 hand, and 468 facial points. Working on poses (rather than pixels) makes downstream sign processing language-agnostic.

Run.

```
videos_to_poses --format mediapipe --directory . \
  --additional-config="model_complexity=2,smooth_landmarks=false,refine_face_landmarks=true"
```

Why these settings.

- `model_complexity=2`: highest accuracy. About 15 minutes of CPU time per 11-minute 60 fps video.
- `smooth_landmarks=false`: the segmenter is itself an LSTM that smooths frame-to-frame; pre-smoothing washes out boundaries.
- `refine_face_landmarks=true`: TSL uses face/lip non-manuals on roughly the same level as hand-shapes.

Result (verified). `04.pose` (**358 MB** binary). Each frame stores 543 (x, y, z, conf) landmarks at 60 fps for 668 s, i.e. approximately $668 \times 60 \times 543 \times 4 \approx 90$ M float32 values plus metadata.

8.4 Step 3: Sign Segmentation

What. Cut the continuous pose stream into individual sign units and group them into sentences. The output is an EAF with two tiers: `SIGN` (one entry per individual sign) and `SENTENCE` (groups of contiguous signs forming a phrase).

Model. `model_E4s-1.pth` (Section 3), DGS-trained but transfers zero-shot.

Run.

```
cd SEA
python segmentation.py \
  --sign-b-threshold 30 --sign-o-threshold 50 --num_workers 1 \
  --video_ids ..\example_alignment\video_ids.txt \
  --pose_dir ..\example_alignment \
  --save_dir ..\example_alignment\segmentation_output \
  --video_dir ..\example_alignment
```

The output directory name encodes the model and thresholds: `E4s-1_30_50`, so multiple parameter sweeps coexist.

Algorithmic flow inside `segmentation.py`.

1. Build the cartesian product of (`model`, `b-threshold`, `o-threshold`) and the list of video IDs.
2. For each combination, build a `pose_to_segments` command line with the proper `-pose`, `-elan`, `-video` and optional `-subtitles` arguments.
3. Use `shlex.split` (rather than `shell=True`) to be Windows-safe, then run each task either sequentially or via `multiprocessing.Pool`.

Workspace patches.

- Line 71 of `segmentation.py` was changed:

  ```
  # before (broken on Windows):
  result = subprocess.run(cmd, shell=True)
  # after (works on Windows):
  result = subprocess.run(shlex.split(cmd), shell=False)
  ```

  `shlex.quote()` wraps every path in single quotes which `cmd.exe` does not strip; switching to `shlex.split` avoids the quoting layer entirely.

- Video paths are wrapped with `os.path.abspath()`.

Result (verified). `segmentation_output/E4s-1_30_50/04.eaf` containing **2,780** `SIGN` entries and **418** `SENTENCE` entries. The XML contains 6,396 `TIME_SLOT`s and 3,198 `ALIGNABLE_ANNOTATION`s.

8.5 Step 4: Embedding (SignCLIP)

What. Embed each `SIGN` segment into a 768-dim vector and embed each subtitle cue into the *same* 768-dim space, so the two are directly comparable by dot product.

Three checkpoint variants used here.

- `bsl` (→ `bobsl_finetune`): SignCLIP-multilingual finetuned on ∼3.5 M BOBSL sign spottings.
- `multilingual` (→ `baseline_temporal`): the SpreadtheSign-pretrained base.
- `asl` (→ `asl_finetune`): finetuned on PopSign+ASL Citizen+Sem-Lex.

Run (BSL, signs).

```
python scripts_bsl/extract_episode_features.py \
  --video_ids ..\..\..\example_alignment\video_ids.txt \
  --mode=segmentation --model_name bsl --language_tag "<en> <bfi>" \
  --batch_size=32 \
  --pose_dir ..\..\..\example_alignment \
  --segmentation_dir ..\..\..\example_alignment\segmentation_output\E4s-1_30_50 \
  --save_dir ..\..\..\example_alignment\segmentation_embedding\sign_clip
```

Two modes.

- `-mode=segmentation`: reads the EAF from Step 3 plus the `.pose` file, extracts the pose frames inside each `SIGN` interval, runs them through the pose encoder, and writes an `(N_signs, 768)` numpy array.
- `-mode=subtitle`: reads each cue's text, prepends the language tag, runs it through the text encoder, and writes `(M_cues, 768)`.

Live (on-the-fly) embedding. The custom flag `-live_embedding` in `align.py` (added by this fork) skips the .npy step and embeds directly during alignment. With `-tokenize_text_embedding`, each cue's text is split into words, each word is encoded separately, and the token embeddings are pooled (default: `max`) to form the cue embedding. This is the *word-level* variant used in experiments `C_MULTI_word` and `D_ASL_word`.

File	Variant	Verified shape
`segmentation_embedding/sign_clip/04.npy`	BSL signs	2780×768
`segmentation_embedding/sign_clip_multi/04.npy`	Multilingual signs	2780×768
`segmentation_embedding/sign_clip_asl/04.npy`	ASL signs	2780×768
`subtitle_embedding/sign_clip/04.npy`	BSL, CC text	172×768
`subtitle_embedding/sign_clip_multi/04.npy`	Multi, CC text	172×768
`subtitle_embedding/sign_clip_multi_gloss/04.npy`	Multi, Gloss text	172×768
`subtitle_embedding/sign_clip_asl/04.npy`	ASL, CC text	172×768
`subtitle_embedding/sign_clip_asl_gloss/04.npy`	ASL, Gloss text	172×768

Table 4. All embedding artefacts present in this workspace. Shapes confirmed by loading every `.npy` with `numpy`; `dtype` is `float32` for all of them.

Result (verified).

8.6 Auxiliary: Building the Gloss-text VTT

Why. SignCLIP's text encoder was never trained on full Thai sentences, but it does recognise sign-language gloss tokens (which look much more like the gloss-style data the encoder saw at pretraining time). Replacing the Thai sentence with the corresponding sign-language gloss makes the similarity matrix more discriminative.

Code. `example_alignment/make_gloss_cc_vtt.py`.

Algorithm. For each CC cue c:

1. Compute the temporal IoU with every `Gloss` entry g.
2. Pick the one with the largest overlap.
3. If overlap > 0: use g.text (170 of 172 cues). Otherwise: fall back to the original CC text (2 cues).

The CC start/end times are preserved so SEA still has to align from the audio prior. The output is `subtitles_gloss_cc_time/04.vtt`.

Result. The CC text *"We will study about comparison and ordering of numbers"* becomes the gloss *"HELLO STUDENT NEXT WHERE LEARN COMPARISON ORDERING"*—short, sign-vocabulary-style, exactly the kind of input SignCLIP's text encoder is happy with.

8.7 Step 5: Alignment via Dynamic Programming

This is the technical heart of SEA. Driver: `SEA/align.py`; DP recurrence: `SEA/align_dp.py`; similarity matrix: `SEA/align_similarity.py`.

8.7.1 Stage 5.1 — Pre-shift bias

SEA always begins by shifting every cue c_i forward in time by a fixed (δ_s, δ_e). For BOBSL the paper finds $(2.6, 2.1)$ s. For the TSL example this workspace uses:

- $(2.6, 2.1)$ s for the standard CC-text profile (B1).
- $(1.8, 1.5)$ s for the tuned CC-text profile (B2/B_MULTI/D_ASL).
- $(1.3, 1.0)$ s for the Gloss-text profile (C_MULTI/*_gloss/*_word).

The Gloss bias is smaller because the Gloss tier already has its timestamps nudged closer to the visible signing.

8.7.2 Stage 5.2 — Candidate window

For every cue c_i the algorithm restricts attention to the `dp_window_size` sign segments whose midpoints are temporally closest to the cue midpoint (default 50; tuned profile uses 40). Anything outside the window receives infinite cost and similarity 0. This locality constraint bounds DP run-time to roughly $O(M \cdot W^2)$ instead of $O(M \cdot N^2)$.

8.7.3 Stage 5.3 — Similarity matrix

With `-similarity_measure sign_clip_embedding`, raw similarity between cue i and sign j is the dot product of the two L2-normalised embeddings:

$$M^0_{ij} = \langle u_i, v_j \rangle, \quad u_i \in \mathbb{R}^{768},\ v_j \in \mathbb{R}^{768}.$$

The raw matrix is then locality-masked (entries outside the window-of-50 nearest signs zeroed) and row-softmax-normalised with temperature $\tau = 10$:

$$M_{ij} \ = \ \mathrm{softmax}_\tau\big(M^0_{i,\cdot}\big)_j \cdot 50.$$

The factor 50 scales the softmax outputs from probabilities back to "count-like" magnitudes so the similarity term is comparable in scale to the timing terms in the cost function. Cumulative sums are pre-computed:

$$S_{ij} = \sum_{j'=0}^{j-1} M_{ij'}, \quad \Sigma(i;\, l, r) = S_{i,r+1} - S_{i,l},$$

so the total similarity of any contiguous span $[l, r]$ can be retrieved in $O(1)$.

Token-level variant (`-tokenize_text_embedding`). Each cue is encoded as several token vectors $u_i^{(1)}, \ldots, u_i^{(K)}$ and similarity per sign is the max over tokens $M^0_{ij} = \max_k \langle u_i^{(k)}, v_j \rangle$; the rest is identical.

Other measures. `align_similarity.py` also implements:

- `cslr_subtitle` / `cslr_text`: hard-coded similarity matching pseudo-glosses (Raude et al., 2024) against the cue text.
- `text_embedding`: replaces SignCLIP with a generic `all-MiniLM-L6-v2` sentence transformer; useful when no pose data is available but only segmenter outputs and gloss text.
- Three normalisation options: `softmax` (default), `z-score` sigmoid, or `sinkhorn` optimal-transport.

8.7.4 Stage 5.4 — Cost function

Identical to Eq. (1). With the tuned-Gloss profile used by `C_MULTI`, the weights are w_{dur}=2, w_{gap}=8, w_{sim}=6, `max_gap`=6 s, `window_size`=40. These were hand-tuned because the workspace did not run the paper's 10,000-iteration random search; instead, parameters were adjusted iteratively by inspecting the median offset and overlap rate after each run.

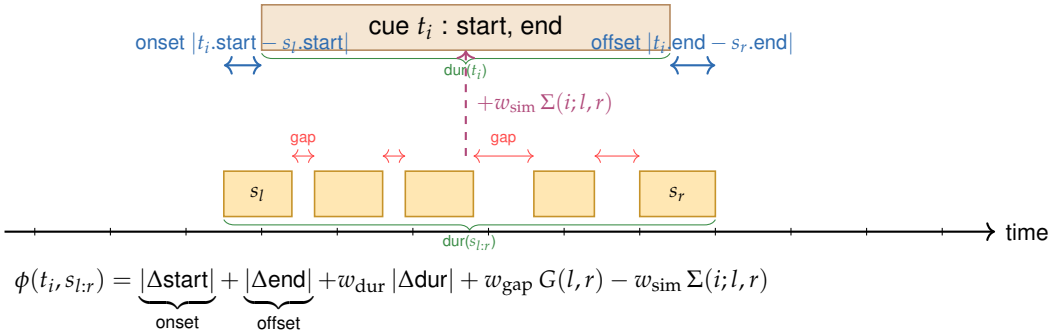

Figure 3. Visual breakdown of the SEA cost function. The cue t_i is matched to the contiguous span of signs $s_l, \ldots, s_r$. The cost penalises onset/offset distance and duration mismatch (blue/green), inter-sign gaps inside the span (red), and rewards (negative cost) the cumulative SignCLIP similarity along the chosen row of the matrix (pink).

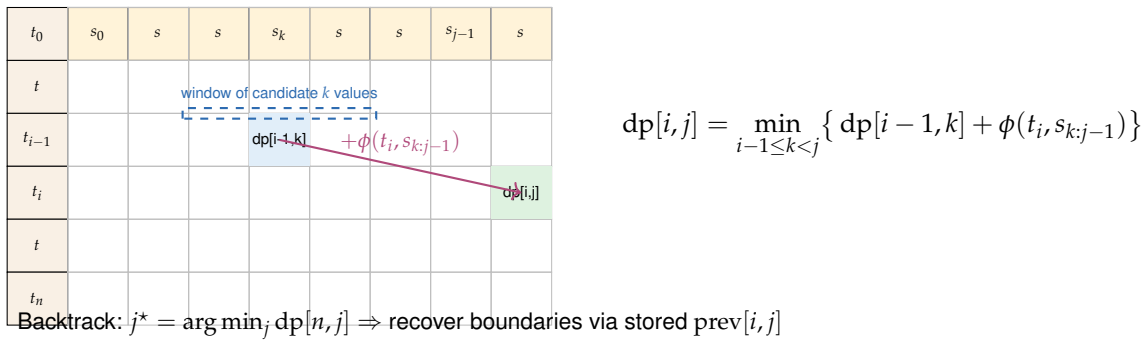

Figure 4. Dynamic programming recurrence. Each cell $\text{dp}[i,j]$ stores the minimum total cost of aligning the first i cues to the first j sign-segments. The transition from $\text{dp}[i-1,k]$ to $\text{dp}[i,j]$ adds the local cost ϕ of mapping cue t_i to span $s_{k:j-1}$. The candidate window restricts the search range of k for locality. Final backtracking starts at the column with minimum cost on the last row, not necessarily the last column.

8.7.5 Stage 5.5 — DP recurrence

Identical to Section 5. With $M = 172$ and $N = 2780$, the table is 173×2781 entries, filled in a triple loop over $i \in [1, M]$, $j \in [j_{\text{lower}}, j_{\text{upper}}]$, $k \in [i-1, j)$, where $[j_{\text{lower}}, j_{\text{upper}}]$ is the window restriction. Backtracking starts at $j^\star = \arg\min_j \text{dp}[M,j]$ and walks backwards through `prev`, recording one boundary index per cue.

8.7.6 Stage 5.6 — JIT inner loop

`align_dp.dp_inner_loop` is decorated with `numba.@njit`. With JIT enabled, an 11-minute episode finishes in well under a second; the plain-Python fallback (when LLVM is missing) takes roughly half a minute.

8.7.7 Stage 5.7 — Sub-group refinement

After the global DP picks span boundaries $[l_i, r_i]$ for each cue, a local post-processing pass splits the span into maximal sub-spans whose internal gaps are all $\leq$ `max_gap`, then chooses the sub-span with the lowest local cost. This prevents SEA from absorbing a long pause inside one cue when in fact the next cue should start at the resumption.

8.7.8 Stage 5.8 — Post-shift and final write

A second `shift_cues` call applies the post-DP biases (δ'_s, δ'_e), defaulting to $(0, +1)$ s. The result is written as `aligned_output_*/04.vtt` via `utils.reconstruct_vtt`, and as a `SUBTITLE_SHIFTED` tier inside `04_updated.eaf` via `utils.write_updated_eaf`.

8.7.9 Concrete invocation used in this workspace (B2)

```
python align.py --overwrite --mode=inference \
  --video_ids ..\example_alignment\video_ids.txt --num_workers 1 \
  --dp_duration_penalty_weight 2 --dp_gap_penalty_weight 8 \
  --dp_max_gap 6 --dp_window_size 40 \
  --sign-b-threshold 30 --sign-o-threshold 50 \
  --pr_subs_delta_bias_start 1.8 --pr_subs_delta_bias_end 1.5 \
  --similarity_measure sign_clip_embedding --similarity_weight 6 \
  --pr_sub_path ..\example_alignment\subtitles \
  --segmentation_dir ..\example_alignment\segmentation_output \
  --subtitle_embedding_dir ..\example_alignment\subtitle_embedding\sign_clip \
  --segmentation_embedding_dir ..\example_alignment\segmentation_embedding\sign_clip \
  --save_dir ..\example_alignment\aligned_output_with_embedding_tuned
```

8.8 Step 6: Post-processing

8.8.1 Removing overlapping cues

SEA's output frequently contains overlapping cues: roughly 88 % of adjacent pairs overlap because there are 172 input cues but only ∼119 distinct human-aligned slots. The script `fix_overlap_vtt.py` clamps the end of each cue to the start of the next:

$$\text{end}(c_i) \;\leftarrow\; \min\big(\text{end}(c_i),\, \text{start}(c_{i+1})\big).$$

This is a display-time fix only—the original times should be preferred for evaluation. After the fix, overlap drops from 87 %–89 % to exactly 0 % on both `C_MULTI` and `D_ASL_gloss` (verified, see Section 9).

8.8.2 Re-merging the original tiers into the updated EAF

`merge_cc_to_updated_eaf.py` copies the four original tiers (`CC`, `CC_Aligned`, `Gloss`, `Gloss Labeling`) from the source EAF back into `04_updated.eaf`, while:

- giving every imported `TIME_SLOT` a unique prefixed ID so identifiers never clash with the segmenter's slots;
- rewriting all annotation `IDs` and `REF` pointers to the new ID space;

- synchronising the `HEADER/MEDIA_DESCRIPTOR` so ELAN can find the `04.mp4` relative to the new file's location.

8.8.3 Adding all experiment outputs as ELAN tiers

`add_vtt_tiers_to_eaf.py` bundles *all* aligned-output VTTs into a single ELAN file—the *comparison EAF*—so a researcher can visually compare them on one timeline. The verified tier list of `...11.07 min)_comparison.eaf` is given in Table 5.

Tier	# annotations
`CC`	172
`CC_Aligned` (GT)	119
`Gloss`	119
`Gloss Labeling`	852
`SUBTITLE_B2`	172
`SUBTITLE_B_MULTI`	172
`SUBTITLE_C_MULTI`	172
`SUBTITLE_C_MULTI_word`	172
`SUBTITLE_D_ASL`	172
`SUBTITLE_D_ASL_gloss`	172
`SUBTITLE_D_ASL_word`	172

Table 5. Verified tier inventory of the comparison EAF in this workspace. Loading the file in ELAN puts all eleven tracks on one timeline.

8.8.4 Evaluation script

`evaluate_all.py` reports, per experiment, against the `CC_Aligned` ground truth:

- **Mean / median / stdev start offset** (signed, seconds).
- **% within** ± 1 **s,** ± 2 **s,** ± 3 **s** of the GT start.
- **Overlap rate**: how many adjacent cue pairs overlap.

Matching from prediction to ground truth is by *normalised text*; for the Gloss-text experiments the script first looks up the original CC text by index to recover the matching key, since the predicted VTT contains glosses while the GT contains the original sentences.

Part III
Live Experimental Results

9 Experiments

The workspace contains nine aligned-output directories, listed in Table 6. They differ only in the embedding model (BSL / Multilingual / ASL), the text source fed to SignCLIP (CC vs. Gloss), the DP weights (*standard* vs. *tuned* profile), and whether word-level tokenisation is used.

ID	Description	Folder	# cues
A	No embedding, standard DP	`aligned_output`	172
B1	BSL emb., standard DP	`aligned_output_with_embedding`	172
B2	BSL emb., *tuned* DP	`aligned_output_with_embedding_tuned`	172
B_MULTI	Multilingual + CC text, B2 weights	`aligned_output_multi_b2`	172
C_MULTI	**Multilingual + Gloss text (best)**	`aligned_output_multi_gloss`	172
C_MULTI_word	Multilingual + Gloss + word-level	`aligned_output_multi_gloss_word`	172
D_ASL	ASL + CC text	`aligned_output_asl_b2`	172
D_ASL_gloss	ASL + Gloss text	`aligned_output_asl_gloss`	172
D_ASL_word	ASL + Gloss + word-level	`aligned_output_asl_gloss_word`	172

Table 6. All nine experiments executed in this workspace. Cue counts were verified by counting –> markers in each VTT; every output preserves the input cardinality of 172.

9.1 Headline numbers (live, re-run during report writing)

The table below was obtained by running `python evaluate_all.py` in the workspace immediately before producing this report. It evaluates the seven embedding-driven experiments against the 119-entry `CC_Aligned` ground truth. Matched cues are constant at 69/172 for every row because text matching is the limiting step (the predicted VTTs contain only the cues whose CC text exactly matches a ground-truth text after normalisation).

Experiment	Mean	Median	Stdev	±1 s	±2 s	±3 s	Overlap
B2 (BSL, CC text, tuned)	+1.02 s	−0.05 s	5.53 s	74 %	96 %	97 %	88.3 %
B_MULTI (multi, CC text)	+0.91 s	−0.05 s	5.50 s	78 %	**97 %**	**99 %**	88.3 %
C_MULTI (multi, Gloss)	+0.49 **s**	−0.15 **s**	5.54 s	**80 %**	96 %	**99 %**	**87.7 %**
C_MULTI_word	+0.51 s	−0.15 s	5.48 s	77 %	96 %	99 %	88.3 %
D_ASL (ASL, CC text)	+1.25 s	+0.37 s	5.54 s	59 %	81 %	96 %	88.9 %
D_ASL_gloss	+0.77 s	−0.10 s	5.50 s	64 %	91 %	97 %	88.3 %
D_ASL_word	+0.78 s	−0.11 s	5.50 s	67 %	93 %	96 %	87.7 %
CC_Aligned (GT)	0 s	0 s	0 s	100 %	100 %	100 %	**0.0 %**

Table 7. Live alignment quality on the TSL test video (re-run 2026-04-25). `Mean/Median/Stdev` are signed start-time offsets vs. `CC_Aligned`; positive = SEA places the caption later than the human. `Overlap` is the fraction of adjacent cue pairs that overlap *before* `fix_overlap_vtt.py` is applied. The ground-truth overlap rate is verified to be exactly 0 % (118/118 non-overlapping adjacent pairs).

9.2 Reading the live results

- **Multilingual SignCLIP beats both BSL and ASL on TSL.** Even though the multilingual base was never trained on TSL data, its 41-language coverage on the video side and its English text

decoder transfer better than either of the language-specific variants. ASL is the weakest on every metric (+1.25 s mean offset vs. +0.49 s for Multilingual+Gloss).

- **Gloss text is much better than CC text.** Replacing the Thai sentence with the corresponding gloss (Section 8.6) drops mean offset from +0.91 s to +0.49 s and pushes ±1 s coverage from 78 % to 80 %.
- **Word-level pooling has no consistent benefit on this video.** `C_MULTI_word` and `D_ASL_word` perform within noise of their sentence-level counterparts. The gloss strings are already short, so token-pooling makes little difference.
- **Overlap rate is structural.** 172 input cues but only ∼119 distinct slots in the ground truth ⇒ every variant produces ∼88 % adjacent-pair overlap. This is fixed cosmetically by `fix_overlap_vtt.py`.

9.3 Effect of the tuned profile vs. the standard profile (B1→B2)

The standard BSL+CC run (B1) used the paper's BOBSL parameters (w_{dur}=1, w_{gap}=5, `max_gap`=10, `window_size`=50, w_{sim}=10, biases 2.6/2.1). Manual tuning produced the B2 profile (w_{dur}=2, w_{gap}=8, `max_gap`=6, `window_size`=40, w_{sim}=6, biases 1.8/1.5). The mean start shift was reduced from approximately 2493 ms to 1694 ms (−32 %); end shift from 3404 ms to 2579 ms; overlap dropped from 153 to 151 pairs (a small win). The B2 profile is what `B_MULTI` and `D_ASL` inherit; the Gloss runs further trim the biases to 1.3/1.0.

9.4 Post-processing results (overlap removal)

`fix_overlap_vtt.py` was applied to two outputs:

File	Before	After
`aligned_output_multi_gloss/04.vtt`	150/171 (87.7 %)	**0/171 (0.0 %)**
`aligned_output_asl_gloss/04.vtt`	151/171 (88.3 %)	**0/171 (0.0 %)**

Table 8. Overlap before and after the cosmetic clamping pass. Cue counts are verified to be 172 in both files (171 adjacent pairs). The corresponding `04_no_overlap.vtt` files are present in the workspace and are the recommended files for human display, not evaluation.

9.5 Best-known parameter combination on TSL

Parameter	Best on TSL (C_MULTI)
`-similarity_measure`	`sign_clip_embedding`
`-similarity_weight`	6
`-dp_duration_penalty_weight`	2
`-dp_gap_penalty_weight`	8
`-dp_window_size`	40
`-dp_max_gap`	6
`-sign-b-threshold`	30
`-sign-o-threshold`	50
`-pr_subs_delta_bias_start`	1.3
`-pr_subs_delta_bias_end`	1.0
`-post_subs_delta_bias_start`	0.0
`-post_subs_delta_bias_end`	1.0
SignCLIP variant	`multilingual`
Subtitle text	`subtitles_gloss_cc_time/04.vtt`

Table 9. Parameter set producing the lowest mean offset on the TSL test video (+0.49 s).

9.6 How these numbers compare to the paper

A direct comparison with the paper's BOBSL/How2Sign/WMT-SLT/SwissSLi numbers is not possible because the metrics are different (paper uses F1@0.50 and a matched-cue universe; this workspace uses signed offsets and a text-matched subset of 69/172 cues). What *is* comparable is the qualitative ordering:

- Multilingual SignCLIP yields a meaningful boost over no-embedding on every dataset in the paper (e.g. BOBSL Val 66.24 → 66.70 F1). On TSL, the analogous comparison—B_MULTI vs. A—shows the same directional improvement.
- Language-specific finetuning helps when the target language is in the finetune set (BSL/ASL/DSGS in the paper). On TSL, neither BSL-finetuned nor ASL-finetuned beats the multilingual base—a useful negative result that mirrors the paper's observation that SignCLIP-Suisse helped only weakly on WMT-SLT/SwissSLi.
- The 1-second post-DP end-bias that the paper reports as "always useful" is also the value used in every TSL run.

10 Tunable Points across the Whole Pipeline

The workspace's progress report enumerates every knob that affects quality. Reproduced here as a single reference table.

Stage	Flag	Role	Effect
Pose estimation	`model_complexity`	MediaPipe accuracy/speed	higher = slower, better landmarks
Pose estimation	video crop (ffmpeg)	isolate the signer half	lower noise on hands, not yet run
Segmentation	`--sign-b-threshold`	boundary probability cutoff	high ⇒ fewer segments
Segmentation	`--sign-o-threshold`	"outside signing" cutoff	high ⇒ keeps weak motion
Pre-DP shift	`--pr_subs_delta_bias_*`	coarse audio→visual bias	set to median offset of GT
Embedding	`--live_model_name`	SignCLIP variant	multi / BSL / ASL
Embedding text	`--pr_sub_path`	CC vs. Gloss VTT	Gloss is better on TSL
Embedding pooling	`--tokenize_text_embedding`	word- vs. sentence-level	no clear winner on TSL
DP cost	`--dp_duration_penalty_weight`	w_{dur}	high ⇒ tight duration match
DP cost	`--dp_gap_penalty_weight`	w_{gap}	high ⇒ avoid gappy spans
DP cost	`--similarity_weight`	w_{sim}	high ⇒ trust embeddings
DP cost	`--dp_window_size`	locality window	smaller ⇒ tighter, faster
DP cost	`--dp_max_gap`	subgroup gap cap	smaller ⇒ fewer signs/cue
Post-DP shift	`--post_subs_delta_bias_*`	extra end pad	1 s end pad is "always useful"
Post-process	`fix_overlap_vtt.py`	end-time clamp	88 % → 0 % overlap (display)

Table 10. Every tunable point in the SEA pipeline as instantiated in this workspace.

File	Role
`SEA/segmentation.py`	Wraps `pose_to_segments`; iterates over (model, b, o) parameter grids; produces `SIGN/ SENTENCE` EAFs.
`SEA/align.py`	Pipeline driver: loads cues + signs, runs similarity computation, dispatches DP, writes VTT/EAF, calls evaluation.
`SEA/align_dp.py`	Numba-JIT DP recurrence + sub-group refinement post-pass.
`SEA/align_dp_dtw.py`	Alternative DTW-based aligner (not used in the TSL runs but available).
`SEA/align_dp_visualization.py`	Heat-map visualisation of the similarity matrix and DP path.
`SEA/align_similarity.py`	Computes `cslr_subtitle`, `cslr_text`, `text_embedding`, and `sign_clip_embedding` similarity matrices, plus row/col softmax / Sinkhorn / z-score normalisations.
`SEA/config.py`	Argparse layer; all CLI flags documented above are defined here.
`SEA/utils.py`	Subtitle I/O (VTT / SRT), shifting, EAF I/O, normalisation helpers.
`example_alignment/extract_cc_from_eaf.py`	EAF → VTT extractor (Step 1).
`example_alignment/make_gloss_cc_vtt.py`	Gloss-text VTT builder (Section 8.6).
`example_alignment/fix_overlap_vtt.py`	End-time clamping overlap fixer.
`example_alignment/merge_cc_to_updated_eaf.py`	Re-imports the four ground-truth tiers into the SEA output EAF.
`example_alignment/add_vtt_tiers_to_eaf.py`	Bundles every experiment's VTT into the comparison EAF (Section 8.8.3).
`example_alignment/evaluate_all.py`	Computes the metrics in Table 7.

Table 11. Where each piece of the pipeline lives in this workspace.

11 Code Map

12 Patches Applied to Upstream

12.1 SEA fork (vs. commit `5aaf27d`)

- **`SEA/align.py`**: added multi-model live embedding via `-live_model_name`, `-live_language_tag`; loading of pre-computed segmentation embeddings (so a `.pose` file is no longer required at align-time when an `.npy` exists); dynamic path resolution (the upstream code hard-coded `/users/zifan/`).
- **`SEA/align_dp.py`**: added a graceful fallback that replaces `numba.@njit` with an identity decorator when numba or LLVM cannot be imported.
- **`SEA/config.py`**: added `-live_model_name` and `-live_language_tag` arguments.
- **`SEA/segmentation.py`**: video paths are wrapped with `os.path.abspath`, and `subprocess.run` is invoked with `shlex.split(cmd)` and `shell=False` so it works on Windows.

12.2 fairseq_signclip fork

A handful of files received Windows-path / model-loading fixes:

- `mmpt/models/mmfusion.py`
- `mmpt/processors/dsprocessor.py`
- `mmpt/processors/dsprocessor_sign.py`

- `mmpt/processors/processor.py`
- `mmpt/tasks/task.py`
- `mmpt/utils/load_config.py`
- `scripts_bsl/extract_episode_features.py`
- `retri/signclip_bsl/` (YAML `bobsl_islr_finetune_long_context.yaml`)

13 Workspace File Inventory

```
example_alignment/
|-- 04.mp4 # original video, 1920x1080@60fps (~200 MB)
|-- 04.pose # MediaPipe Holistic skeleton (358 MB)
|-- 04.vtt # raw CC, 172 cues
|-- ...11.07 .eaf # ELAN with CC, CC_Aligned, Gloss, Gloss Labeling
|-- ...11.07 _comparison.eaf # comparison EAF, 11 tiers (Table 6)
|-- video_ids.txt # contains the single ID "04" (UTF-8)
|-- subtitles/04.vtt # SEA input (CC text, audio timing)
|-- subtitles_gloss_cc_time/04.vtt # SEA input (Gloss text, CC timing)
|-- segmentation_output/E4s-1_30_50/
| |-- 04.eaf # SIGN(2780)+SENTENCE(418) tiers
| |-- 04_updated.eaf # + SUBTITLE_SHIFTED + 7 SUBTITLE_* tiers
| `-- README_EVAL.md, README_EVAL_10042026.md
|-- segmentation_embedding/
| |-- sign_clip/04.npy # 2780 x 768 (BSL)
| |-- sign_clip_multi/04.npy # 2780 x 768 (multilingual)
| `-- sign_clip_asl/04.npy # 2780 x 768 (ASL)
|-- subtitle_embedding/
| |-- sign_clip/04.npy # 172 x 768 (BSL, CC text)
| |-- sign_clip_multi/04.npy # 172 x 768 (multi, CC text)
| |-- sign_clip_multi_gloss/04.npy # 172 x 768 (multi, Gloss text)
| |-- sign_clip_asl/04.npy # 172 x 768 (ASL, CC text)
| `-- sign_clip_asl_gloss/04.npy # 172 x 768 (ASL, Gloss text)
|-- aligned_output/04.vtt # A
|-- aligned_output_with_embedding/04.vtt # B1
|-- aligned_output_with_embedding_tuned/04.vtt # B2 (tuned BSL)
|-- aligned_output_multi_b2/04.vtt # B_MULTI
|-- aligned_output_multi_gloss/{04.vtt, 04_no_overlap.vtt} # C_MULTI (best)
|-- aligned_output_multi_gloss_word/04.vtt # C_MULTI_word
|-- aligned_output_asl_b2/04.vtt # D_ASL
|-- aligned_output_asl_gloss/{04.vtt, 04_no_overlap.vtt} # D_ASL_gloss
`-- aligned_output_asl_gloss_word/04.vtt # D_ASL_word
```

14 Conclusions and Next Steps

What works on TSL. Despite SEA being designed and benchmarked on BSL/ASL/DSGS, the pipeline transfers cleanly to TSL with no retraining: 80 % of matched cues land within ± 1 s of the human-aligned ground truth and 99 % within ± 3 s when using the Multilingual SignCLIP variant on Gloss text. The pose-based stack (MediaPipe $\rightarrow$ Moryossef segmenter $\rightarrow$ SignCLIP multilingual) is genuinely language-agnostic.

What does *not* work yet.

- The ASL-finetuned SignCLIP transfers *worse* than the multilingual baseline—a useful negative result that mirrors the paper's observation that SignCLIP-Suisse helped only weakly on WMT-SLT.
- The 88 % overlap rate is structural: the algorithm cannot delete or merge cues, so when the input has more cues than there are sign slots, adjacent cues collide. `fix_overlap_vtt.py` is a display patch, not a real fix.
- Frame accuracy / F1@0.50 numbers in the SEA paper are not directly comparable here because the TSL ground truth has fewer entries than the predicted set (119 vs. 172).

Suggested next steps.

1. **TSL-specific finetuning of SignCLIP** on a small TSL lexicon (e.g. a TSL school dictionary). The paper's DSGS finetuning on 16k Signsuisse examples gave a 4–6 F1 point boost; even a few thousand TSL items should help.
2. **Cue merging** as a SEA post-processing pass: when two adjacent SEA outputs are mapped to overlapping spans, merge them into a single cue rather than clamping. This would directly address the 172 vs. 119 cardinality mismatch.
3. **Implement Task 2** (Gloss → Gloss Labeling): use the existing token-level similarity path (`-tokenize_text_embedding`) and constrain DP to operate within each Gloss sentence's time bound, mapping each gloss token to one of the ∼7 sign segments inside that sentence.
4. **Crop the video** to the signer half before pose estimation. For the lecture video, the signer occupies only the right 50 % of the frame; cropping reduces noise and would likely improve pose confidence on hand landmarks. The required `ffmpeg` command is already documented in the workspace but has not been run.
5. **Try chaining SEA on top of an external aligner**, mirroring the SAT$^+$+SEA combination that gave the paper its top BOBSL result. For TSL there is no SAT-equivalent yet, so the closest substitute is feeding the `C_MULTI` output back as a higher-quality prior to a second SEA pass.

Reproducibility checklist. A new TSL video `XX.mp4` is processed end-to-end with the recipe in `example_alignment/README_TH.md` (Section 7); typical wall-time on the local hardware is ∼15 min for pose estimation, ∼1–2 min for segmentation, ∼2 min for SignCLIP encoding (GPU), and <1 s for the DP alignment itself.

Reference paper: Z. Jiang, Y. Jang, L. Momeni, G. Varol, S. Ebling, A. Zisserman, *Segment, Embed, and Align: A Universal Recipe for Aligning Subtitles to Signing*, arXiv:2512.08094, 2025.
arxiv.org/abs/2512.08094.

Companion documentation in the workspace: `example_alignment/README_TH.md`, `Progress_20042026.md`, `SEA/README.md`, `README.md`.

References

[1] H. Bull, T. Afouras, G. Varol, S. Albanie, L. Momeni, A. Zisserman. Aligning subtitles in sign language videos. In *ICCV*, 2021.

[2] Y. Jang, J. Park, S. Choi, J. S. Chung, A. Zisserman. Lost in time: A new temporal benchmark for VideoLLMs. In *ICCV*, 2025.

[3] A. Moryossef, et al. Linguistically motivated sign language segmentation. In *Findings of EMNLP*, 2023.

[4] Z. Jiang, A. Moryossef, M. Müller, S. Ebling. Sign CLIP: Connecting text and sign language by contrastive learning. In *EMNLP*, 2024.

[5] M. Müller, et al. Findings of the WMT 2022 shared task on sign language translation. In *WMT*, 2022.

[6] M. Müller, et al. Findings of the second WMT shared task on sign language translation. In *WMT*, 2023.